

TWO POINTERS

"The more we do the more we can do."

~ William Hazlitt



BRIGIT
ADAMS
.com



Good . Evening -

Today's content

- Pair sum
- Pair difference
- Maximum water accumulated
- $\underbrace{\text{Max}(\) - \text{Min}(\)}_{\text{Minimised}}$

$$3 \text{ pointers} \Rightarrow \text{arr}[i] + \text{arr}[j] + \text{arr}[k] = S$$

$$4 \text{ pointers} = \text{arr}[i] + \text{arr}[j] + \text{arr}[k] + \text{arr}[l] = S$$

Pointer $\rightarrow$ variable that stores index value

Two pointer Approach $\rightarrow$ If we are using 2 variables to store indices & with the help of these 2 ptr, if we can traverse the complete DS then that is 2 pointer approach.

2 pointer approach $\rightarrow$ Array reversing
 $\rightarrow$ Sliding window
 $\rightarrow$ Mergesort / Quicksort

Q1. Given arr[N] distinct sorted elements, check if there exists a pair (i, j) such that $arr[i] + arr[j] = k$ & $i \neq j$

Eg:- $arr[5] = \{ \underset{0}{3} \ \underset{1}{7} \ \underset{2}{8} \ \underset{3}{12} \ \underset{4}{19} \}$ $k=15$

Brute force:-

① Check all pairs $\rightarrow$ $TC = O(n^2)$
 $SC = O(1)$

② Optimise

(a) Use hashing technique

$TC: O(n)$
 $SC: O(n)$

Constraint :- No extra space allowed

(b) Use binary search

Idea $\rightarrow$ Fixing one element & search for
the counterpart in array using BS

$\downarrow$
 $arr = \{2, 6, 8, 10\} \quad k = 13$

$$a + b = 13$$

$a = 2$, search for 11

$a = 6$, search for 7

$a = 8$, search for 5

$a = 10$, search for 3

TC : $O(n \log n)$

SC : $O(1)$

04. Two pointers

$arr[] =$

-3	0	1	3	6	8	11	14	18	25
----	---	---	---	---	---	----	----	----	----

 $k = 17$

$\uparrow \qquad \qquad \qquad \uparrow$
 $P_1 \qquad \qquad \qquad P_2$

P_1	P_2	$A[P_1] + A[P_2] = 17$	
0	9	$-3 + 25 = 22 > 17$	decrease sum P_2--
0	8	$-3 + 18 = 15 < 17$	inc sum, P_1++ :
1	8	$0 + 18 = 18 > 17$	dec sum, P_2-- :
1	7	$0 + 14 = 14 < 17$	inc sum, P_1++ :
2	7	$1 + 14 = 15 < 17$	inc sum, P_1++ :
3	7	$3 + 14 = 17 == 17$	True

boolean pairsum (int [] arr, int k)

$P_1 = 0$ $P_2 = n - 1$

while ($P_1 < P_2$)

if ($arr[P_1] + arr[P_2] == k$) {

 return True;

}

else if ($arr[P_1] + arr[P_2] < k$) {

$P_1 = P_1 + 1$

}

else if ($arr[P_1] + arr[P_2] > k$) {

$P_2 = P_2 - 1$;

}

}

return false;

TC: $O(N)$

SC: $O(1)$

Why?

$$ar[] = \{ \overset{*}{3} \ 9 \ 10 \ 14 \ \overset{*}{18} \} \quad K=19$$

$\uparrow \qquad \qquad \uparrow$
 $P_1 \qquad \qquad P_2$

$$A[P_1] + A[P_2]$$

$$3 + 18 = 21 > 19 \quad - \text{ If } 18 + \text{smallest ele} > 19$$

$$\begin{bmatrix} 9 \\ 10 \\ 14 \end{bmatrix} + \boxed{18} > 19$$

→ then 18 + {all other ele}
are definitely going to be > 19
so we can discard 18

$$A[P_1] + A[P_2]$$

$$\boxed{3} + 14 = 17 < 19 \quad - \text{ If } 3 + \text{largest ele} < 19$$

$$3 + \begin{bmatrix} 9 \\ 10 \end{bmatrix} = < 19 \rightarrow 3 \text{ paired up with all other}$$

ele is definitely going to be less than 19, so we can discard 3

02. Given an array of sorted elements, check if there exists a pair (i, j) such that $ar[j] - ar[i] = k$, $i \neq j$ & $k \geq 0$

$arr[]$	=	<table><tr><td>-3</td><td>0</td><td>1</td><td>3</td><td>6</td><td>8</td><td>11</td><td>14</td><td>18</td><td>25</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table>	-3	0	1	3	6	8	11	14	18	25	0	1	2	3	4	5	6	7	8	9	$k=5$
-3	0	1	3	6	8	11	14	18	25														
0	1	2	3	4	5	6	7	8	9														
		$\uparrow$ P_1 $\uparrow$ P_2																					

Case 1 = $\{0, n-1\}$ $\longrightarrow$ Ambiguity, no clear picture for movement P_1 & P_2

P_1	P_2	$A[P_2] - A[P_1]$
0	9	$25 - (-3) = 28 > k$

— Look for better initialisation

decrease diff

P_1++ diff = 25

P_2-- diff = 21

Case - II = $\{mid, mid+1\}$ $\longrightarrow$ Ambiguity, we can't decide

P_1	P_2	$A[P_2] - A[P_1]$
4	5	$8 - 6 = 2 < 5$

increase diff

P_2++ diff = 5

P_1-- diff = 5

Case - III $\{0, \text{mid}\}$ $\rightarrow$ Ambiguity. can't decide

P_1 P_2 $A[P_2] - A[P_1]$

0 4 $6 - (-3) = 9 > 5$

decrease diff

P_1++ : diff = 6

P_2-- : diff = 6

$arr[] =$

-3	0	1	3	6	8	11	14	18	25
0	1	2	3	4	5	6	7	8	9

 $k=5$

$\uparrow$ $\uparrow$
 P_1 P_2

Case IV $\{0, 1\}$

Case V $\rightarrow \{n-1 \ \& \ n-2\}$ {TODO}

P_1 P_2 $A[P_2] - A[P_1]$

0 1 $0 - (-3) = 3 < 5$, inc diff , P_2++

0 2 $1 - (-3) = 4 < 5$, inc diff , P_2++

0 3 $3 - (-3) = 6 > 5$, dec diff , P_1++

1 3 $3 - 0 = 3 < 5$, inc diff , P_2++

1 4 $6 - 0 = 6 > 5$, dec diff , P_1++

2 4 $6 - 1 = 5 = 5$ return True

boolean diff (int ar[], int k)

$k = \text{abs}(k)$

$P_1 = 0$ $P_2 = 1$

while ($P_1 < n$ && $P_2 < n$)

if ($A[P_2] - A[P_1] == k$) {

return true;

}

else if ($A[P_2] - A[P_1] > k$) {

$P_1 = P_1 + 1$

if ($P_1 == P_2$) $P_2 = P_2 + 1$

}

else {

$P_2 = P_2 + 1$;

}

}

return false;

$ar[P_2] - ar[P_1] = k$

$ar[P_1] - ar[P_2] = -k$

3

Eg:- $arr = \{4, 10, 13\}$ $K=0$

$\begin{array}{ccc} \underset{0}{4} & \underset{1}{10} & \underset{2}{13} \\ & \uparrow & \uparrow \\ & P_1 & P_2 \end{array}$

$P_1 \quad P_2 \quad arr[P_2] - arr[P_1]$

0 1 $10 - 4 = 6 > 0$ dec diff, P_1++ :

1 1 $P_1 = P_2 \rightarrow$ explicitly P_2++ :

1 2 $13 - 10 = 3 > 0$ dec diff, P_1++ :

2 2 $P_1 = P_2 \rightarrow$ explicitly P_2++ :

2 3 $\rightarrow$ Stop

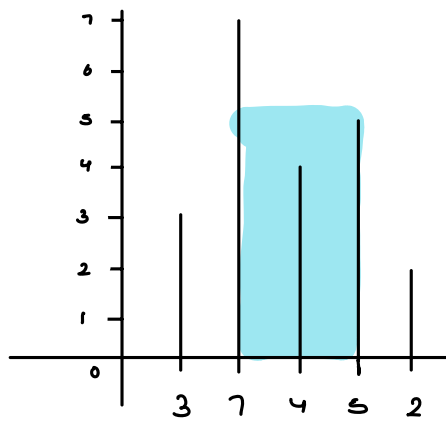
03. Water logging

Given $arr[N]$ elements, $arr[i]$ represents height of each wall. Find max water accumulated between any two walls.

Note:- Between two walls, 1 unit distance is present

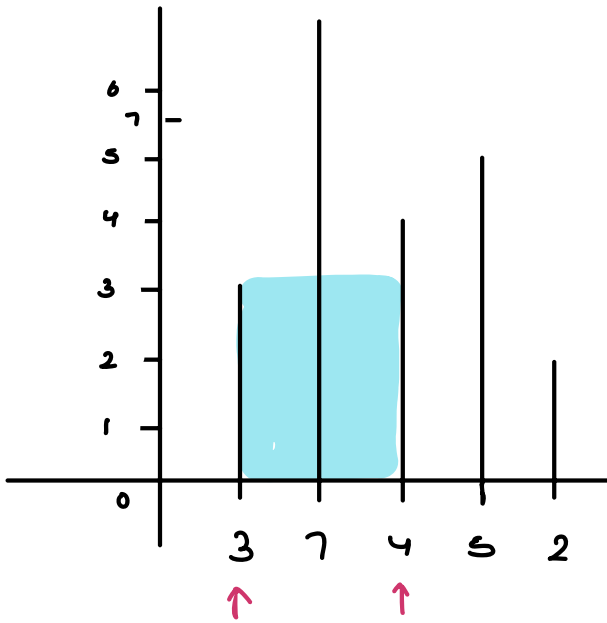
Eg:- $arr[5] = \{3, 7, 4, 5, 2\}$

$\begin{array}{ccccc} & 0 & 1 & 2 & 3 & 4 \end{array}$

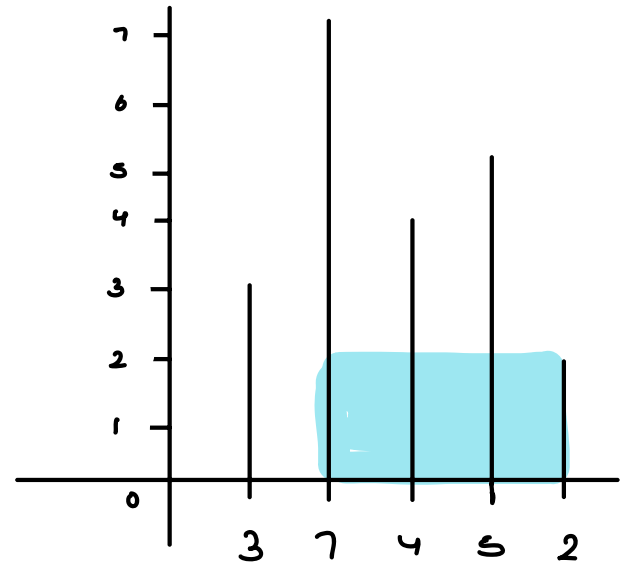


$$Ht = 5, W = 2$$

$$\text{Water} = 10$$

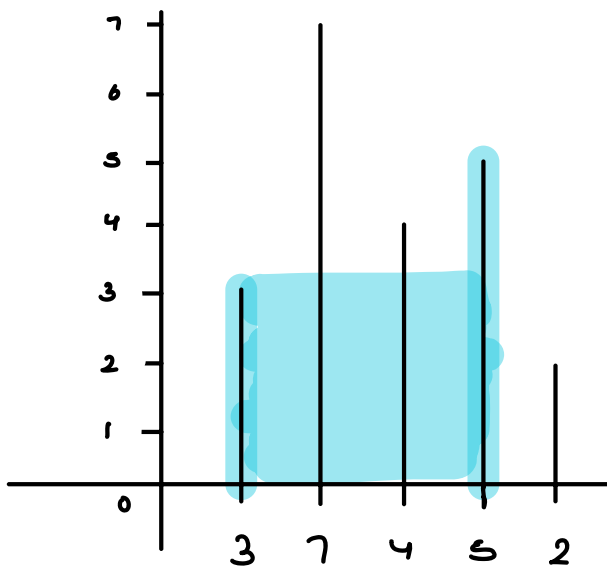


$$\text{Water} = 3 * 2 = 6$$



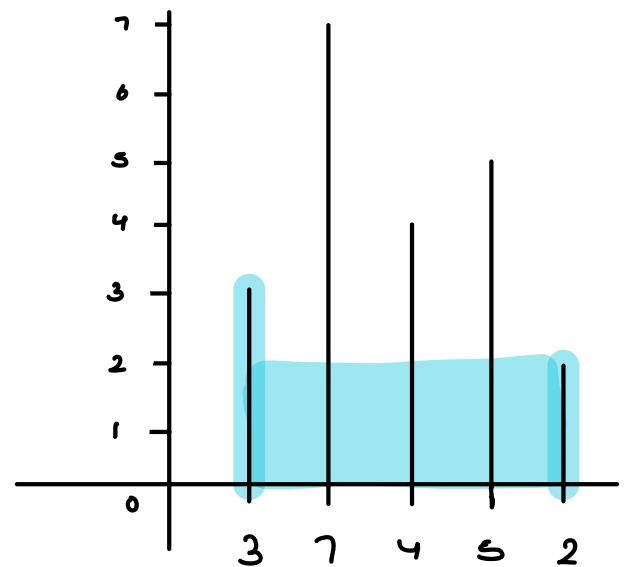
$$Ht = 2, W = 3$$

$$\text{Water} = 2 * 3 = 6$$



$$Ht = 3, W = 3$$

$$\text{Water} = 3 * 3 = 9$$



$$Ht = 2, W = 4$$

$$\text{Water} = 2 * 4 = 8$$

Brute Force → For each & every wall pair, calculate the amount of water getting accumulated & update the maximum

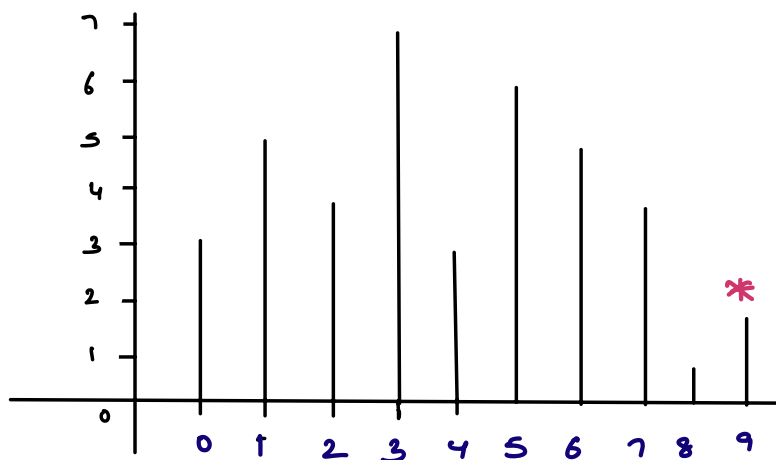
```
int water (int [] ar)
{
    ans = 0
    for (i = 0; i < n; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            int ht = Math.min(A[i], A[j]);
            int w = j - i
            water = ht * w
            ans = Math.max(ans, water)
        }
    }
    return ans
}
```

$Tc: O(n^2)$

$Sc: O(1)$

II Optimised Idea

$ar[10] = \{ 3, 5, 4, 7, 3, 6, 5, 4, 1, 2 \}$



↑
P₁

↑
P₂

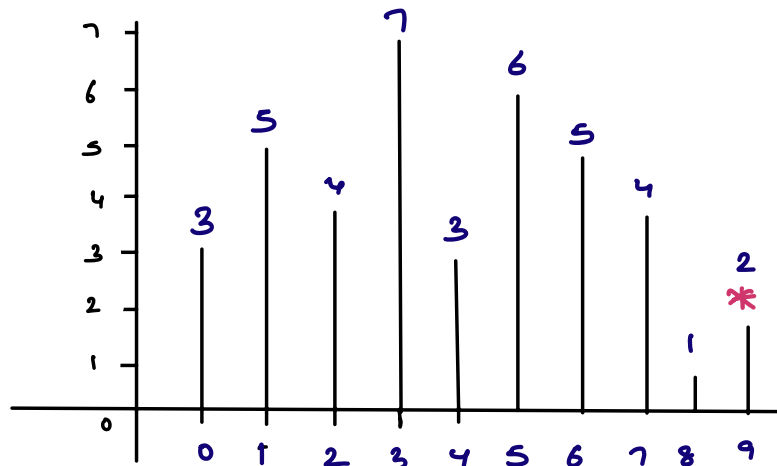
since smaller than A[P₁]

$$H = 2 \quad W = 9 \quad \text{Water} = 18$$

↑
Move P₂ → Fix P₂ & move P₁, then we
can never get better ans than 18

Idea → Move the pointer with small height
to get the better ans.

$$\text{arr}[10] = \{ 3 \ 5 \ 4 \ 7 \ 3 \ 6 \ 5 \ 4 \ 1 \ 2 \}$$



↑ ↑
P₁ P₂

P ₁	P ₂	H = min(A[P ₁], A[P ₂])	W = P ₂ - P ₁	Water	
0	9	2	9	18	P ₂ --
0	8	1	8	8	P ₂ --
0	7	3	7	21	P ₁ ++;

1	7	4	6	24	P_2--
1	6	5	5	25	P_1++
2	6	4	4	16	P_1++
3	6	5	3	15	$P_2--;$
3	5	6	2	12	$P_2--;$
3	4	3	1	3	$P_2--;$

3 3 $\rightarrow$ break

return 25

```
int water (int [] A)
```

```
    P1 = 0    P2 = n - 1    ans = 0
```

```
    while (P1 < P2)
```

```
        int ht = Math.min (A[P1], A[P2])
```

```
        int w = P2 - P1
```

```
        water = ht * w
```

```
        ans = Math.max (ans, water)
```

```
        if (A[P1] ≤ A[P2]) P1 = P1 + 1;
```

```
        else P2 = P2 - 1;
```

```
    }
```

```
    return ans;
```

```
}
```

Two pointer

01. Initialize pointers
02. Discard one pointer $\rightarrow$ If we can't get a better ans using this ptr, discard it
03. Update the pointers inside the loop

Q Given 3 sorted arrays $A[]$, $B[]$, $C[]$ of size N
find i, j, k such that

$$\underbrace{\max(A[i], B[j], C[k]) - \min(A[i], B[j], C[k])}_{\text{is minimized}}$$

	0	1	2	3	4	5
$i \rightarrow A[] =$	3	14	16	20	29	40
$j \rightarrow B[] =$	-6	23	24	30	35	50
$k \rightarrow C[] =$	-15	15	26	31	39	42

i	j	k	$\max(A[i], B[j], C[k]) - \min(A[i], B[j], C[k])$
0	0	0	$3 - (-15) = 18$
0	0	1	$15 - (-6) = 21$ *
5	4	4	$40 - 35 = 5$
4	3	3	$31 - 29 = 2$

Hint → Start from $i = 0$
 $j = 0$
 $k = 0$

ans = 18 → now you have to think
to decrease the ans

Ans → Move the pointer with min value